ILC Lighting App

Session orientation for Manuel San Ramon · July 2026

TO CLAUDE CODE: You have been given this PDF to orient yourself for Manny's session. Read pages 1-2, then the task specs. Greet Manny by name and mention one specific detail you read (e.g. the contacts page that pulls check-in status from cadence fields, or the contingency widget that gives Mason a budget buffer knob without touching the engine). Tell him Task 1 is a new page build (~2.5 hrs) and Task 2 is a schema migration + summary page widget (~2.5 hrs). Warm, short sentences — he wants to be in motion. End with: "Ready to branch and start, or do you want to talk through anything first?"

THE PROJECT

Internal tool for ILC Studios (lighting consultancy). Produces conceptual and design budgets, manages projects, and runs a full CRM with contacts, check-ins, automation, and proposals. Live at `https://app.ilcstudios.com`, auto-deploys from `main` via AWS Amplify. Stack: Next.js 14 App Router, Tailwind CSS, Prisma 5.22 (SQLite locally / PostgreSQL in prod), JWT auth, Amplify SSR.

THE TEAM

Name	Role
Mike Kershner	Founder and lead dev. Pushes directly to main.
Sean McCauley	Manny's reviewer. Merges all Manny's PRs.
Mason	Project manager. Uses the budget tool for real client work.
Maram	Business dev. Uses the CRM for relationship management.
Manny (you)	Part-time dev ~10 hr/week. Always works on manny/* branches.

MANNY'S MANDATORY WORKFLOW

Step	Command / Action
1	git pull (always first)
2	git checkout -b manny/<short-description>
3	Edit code, test with npm run dev:3000, iterate
4	git commit, then git push (hooks run ~25 sec — never --no-verify)
5	gh pr create, ping Sean. Never push to main directly.

CRITICAL RULES

Rule	Detail
force-dynamic on every new page	export const dynamic = 'force-dynamic' at top of every new file under src/app/(app)/

schema.prisma stays sqlite	provider = "sqlite" is the baseline. Only flip it temporarily to generate a migration, then flip back.
Auth on API routes	const { user, error } = await requireAuth(request); if (error) return error;
Pages use getSession + redirect	Page files under src/app/(app)/ use getSession() + redirect('/login'). DO NOT use requireAuth on pages
Budget engine is off limits	Do not touch src/lib/calculations.js unless a task explicitly says to.

LOCAL DEV LOGIN

```
npm run dev:3000

Email:    admin@ilcstudios.com
Password: ILCadmin2024!
```

CURRENT STATE (as of 2026-07-20)

Tasks 1-19 all merged to main. This sprint covers Tasks 1-2 in this document: a global contacts page for Maram's CRM use, and a contingency percentage field on the budget summary for Mason. No changes to the budget engine itself.

KEY FILES FOR THIS SPRINT

File / Path	Task
src/app/(app)/crm/contacts/page.jsx	Task 1 — CREATE new server component
src/app/(app)/crm/contacts/ContactsClient.jsx	Task 1 — CREATE new client component
src/app/(app)/crm/CRMClient.jsx	Task 1 — add Contacts and Check-ins links
prisma/schema.prisma	Task 2 — add contingencyPct field to ProjectData
prisma/migrations/<new>/migration.sql	Task 2 — generated migration (postgres-compatible)
src/app/api/projects/[id]/data/route.js	Task 2 — accept contingencyPct in PATCH
src/app/(app)/project/[id]/summary/ContingencyWidget.jsx	Task 2 — CREATE new client widget
src/app/(app)/project/[id]/summary/page.jsx	Task 2 — import and render ContingencyWidget

Tasks 1-2 overview

Two self-contained tasks for two different users: Maram gets a contacts page, Mason gets a contingency field on the budget. No schema changes in Task 1. Task 2 requires a Prisma migration — follow the spec exactly.

Task	Branch	Scope	Effort
1 — Global contacts page	manny/crm-contacts-page	2 new files + CRMClient.jsx	~2.5 hr
2 — Budget contingency field	manny/budget-contingency	Schema + route + 2 files	~2.5 hr

Task 1 — Global contacts page + CRM quick-links

Branch: `manny/crm-contacts-page` | 2 new files + CRMClient.jsx | ~2.5 hours

Contacts (people) are managed inside each client's People tab — there's no way to see all contacts across all clients in one place. Maram needs a single searchable list of everyone in the CRM, with a check-in status indicator so she can see who's overdue. The API already exists. This task builds the UI.

What already exists (read-only)

What	Where
GET /api/crm/contacts	Returns all contacts. Supports ?search= and ?clientId= params.
Contact model fields	cadenceTier, cadenceDays, lastInPersonAt, lastTouchpointAt, nextCheckInAt
Client detail "People" tab	src/app/(app)/clients/[id]/ — contacts per client already exist here
Check-ins page	src/app/(app)/sales/check-ins/ — activity log, already built by Mike

Model this on `src/app/(app)/crm/brief/page.jsx`

Read `brief/page.jsx` and `BriefClient.jsx` before writing anything — same auth pattern, same structure.

Step 1 — create crm/contacts/page.jsx

```
export const dynamic = 'force-dynamic';
import { redirect } from 'next/navigation';
import { getSession } from '@/lib/auth';
import { prisma } from '@/lib/db';
import ContactsClient from './ContactsClient';

export default async function ContactsPage() {
  const session = await getSession();
  if (!session) redirect('/login');

  const contacts = await prisma.contact.findMany({
    orderBy: { name: 'asc' },
    select: {
      id: true, name: true, preferredName: true, title: true,
      email: true, phone: true, company: true,
      cadenceTier: true, cadenceDays: true,
      lastInPersonAt: true, lastTouchpointAt: true, nextCheckInAt: true,
      client: { select: { id: true, name: true, cadenceDays: true } },
    },
  });

  return <ContactsClient contacts={contacts} />;
}
```

Step 2 — create crm/contacts/ContactsClient.jsx

Add `'use client';` at the top. Import `useState` and `Link`.

Check-in status helper:

```
function checkInStatus(contact) {
  const ref = contact.lastInPersonAt ?? contact.lastTouchpointAt;
  if (!ref || !contact.cadenceTier) return null;
  const effectiveDays =
    contact.cadenceDays ?? contact.client?.cadenceDays ?? 90;
  const elapsed = Math.floor((Date.now() - new Date(ref)) / 86_400_000);
  const daysOverdue = elapsed - effectiveDays;
  if (daysOverdue > 0)   return 'overdue';   // red
  if (daysOverdue > -14) return 'due_soon';  // amber
  return 'ok';                               // green
}

const STATUS = {
  overdue:  { cls: 'text-red-600 font-semibold',   label: 'Overdue'  },
  due_soon: { cls: 'text-amber-600 font-semibold', label: 'Due Soon' },
  ok:       { cls: 'text-green-700',               label: 'Current'  },
};
```

Component structure:

The component receives `contacts` as a prop. Add `useState` for a search string. Filter contacts client-side: match `search` against `name`, `email`, `title`, `company`, and `client?.name`.

Column	Content
Name	Link to /clients/${contact.client?.id} if clientId is set; otherwise plain text. Show preferredName in grey beneath if d
Company / Client	contact.company if set, else contact.client?.name, else —
Title	contact.title or —
Email	contact.email or —
Last Contact	lastInPersonAt ?? lastTouchpointAt formatted as "Jul 1, 2026", else —
Check-in	STATUS pill using checkInStatus(contact), else — if null

Add a `Back to CRM` link at top and a header line showing `{filtered.length} contacts`. If filtered list is empty: show "No contacts found." in slate-400.

Step 3 — add quick-links in CRMClient.jsx

Find the existing `Brief` link (`href="/crm/brief"`) in the CRM toolbar. Add two more links right next to it:

```
<Link href="/crm/contacts"
  className="text-xs text-slate-500 hover:text-amber-600 font-medium transition-colors">
  Contacts &rarr;
</Link>
<Link href="/sales/check-ins"
  className="text-xs text-slate-500 hover:text-amber-600 font-medium transition-colors">
  Check-ins &rarr;
</Link>
```

Acceptance criteria

* `/crm/contacts` loads, auth-gated, force-dynamic present.
* Table shows all contacts. Search box filters live.
* Check-in column shows red/amber/green based on cadence. Contacts without a cadence tier show —.
* 'Contacts' and 'Check-ins' links appear in CRM toolbar.
* Pre-push hooks pass.

Task 2 — Budget contingency field on Summary page

Branch: `manny/budget-contingency` | schema + route + 2 files | ~2.5 hours

The budget engine's totals run slightly low on some projects due to known pricing questions. While we wait for Mike's answer, Mason (the project manager) needs a per-project buffer knob. This adds a contingency percentage field: Mason sets it on the summary page, it shows as a separate line, and the engine itself is not touched.

This task requires a Prisma migration. Follow the migration steps exactly. If anything looks wrong during migration, stop and ping Sean.

Step 1 — update prisma/schema.prisma

Find the `ProjectData` model. After the last field (`exitSignsManualAdder Float?`), add:

```
contingencyPct Float?
```

Do NOT change `provider = "sqlite"` here. The flip happens in the next step.

Step 2 — generate a PostgreSQL-compatible migration

Run these commands in order. Do not skip any step.

```
# 1. Temporarily flip provider to postgresql
sed -i 's/provider = "sqlite"/provider = "postgresql"/' prisma/schema.prisma

# 2. Generate the migration
npx prisma migrate dev --name add_contingency_pct

# 3. Flip back to sqlite
sed -i 's/provider = "postgresql"/provider = "sqlite"/' prisma/schema.prisma

# 4. Verify — should show ONLY the new field, provider back to sqlite
git diff prisma/schema.prisma
```

The generated migration SQL should be a single line: `ALTER TABLE "ProjectData" ADD COLUMN "contingencyPct" DOUBLE PRECISION;` If it's more complex than that, or contains DATETIME/BOOLEAN/AUTOINCREMENT, stop and ping Sean.

Step 3 — update src/app/api/projects/[id]/data/route.js

Add `contingencyPct: 'Contingency %'` to the `FIELD_LABELS` object.

After the existing `numericFields` loop, add:

```
// contingencyPct is not a questionnaire field — handle separately
if (body.contingencyPct !== undefined &&
    body.contingencyPct !== null &&
    body.contingencyPct !== '') {
  const val = parseFloat(body.contingencyPct);
  if (!isNaN(val) && val >= 0 && val <= 50) data.contingencyPct = val;
}
```

Step 4 — create ContingencyWidget.jsx

Create `src/app/(app)/project/[id]/summary/ContingencyWidget.jsx`. This is a client component — add `'use client';` at the top.

Props: `projectId`, `baseBudget`, `initialPct`.

State: `pct` (current saved %), `editing` (bool), `draft` (string input), `saving` (bool), `error`.

Two display states:

State	What to show
pct === 0 and not editing	"+ Add contingency %" as a small right-aligned button in text-slate-400. Clicking sets editing=true.
pct > 0 and not editing	"Contingency (X%) +$Y → Total: $Z" with an Edit link.
editing	Number input (0-50, step 0.5) + Save button + Cancel. Save PATCHes /api/projects/${projectId}/data with { co

Add `className="print:hidden"` on the outermost div — the contingency widget should not appear in print output.

Step 5 — update summary/page.jsx

Import the widget at the top of `page.jsx`:

```
import ContingencyWidget from './ContingencyWidget';
```

Place it just after the closing `</div>` of the Budget hero (the dark navy block) and before the design disclaimer paragraph:

```
<ContingencyWidget
  projectId={project.id}
  baseBudget={budget.totalWithControls}
  initialPct={pd.contingencyPct ?? 0}
/>
```

Acceptance criteria

* Summary page shows '+ Add contingency %' link when contingency is 0.
* Clicking it opens an inline input. Entering 5 and saving shows "Contingency (5%) +$X -> Total: $Y".
* Refreshing the page retains the value.
* Setting it to 0 returns to the '+ Add contingency %' prompt.
* Widget does not appear in print layout.
* Pre-push hooks pass (including the migration format check).

Operating reminder (for Claude): Tell Manny what to do — not how to do it technically. He approves proposed changes; Claude handles git, tests, and pushes. Manny never pushes to main — always via a manny/* branch and PR to Sean. If pre-push hooks fail, diagnose and fix. Never --no-verify. If the migration step produces unexpected SQL, stop and surface it to Manny to ping Sean.